

Advances in Financial Machine Learning: A Critical Implementation Study of López de Prado's Framework on Cryptocurrency Microstructure Data

Abstract

This report presents a critical implementation study of the methodological framework introduced in Marcos López de Prado's *Advances in Financial Machine Learning* (AFML, 2018). We implement the core pipeline—information-driven dollar bars, fractional differentiation, triple-barrier labeling, sample uniqueness weighting, purged cross-validation, meta-labeling, probability calibration, dynamic bet sizing, and deflated Sharpe ratio analysis—on high-frequency Binance SOL/USDT aggregated trade data for October 2023. The pipeline transforms 3,105,897 tick records into 4,549 dollar bars and proceeds through feature engineering, labeling, model training, and out-of-sample walk-forward validation. Our findings illustrate that the AFML framework's primary value lies not in any single technique but in the systematic addressing of financial machine learning's unique challenges: non-stationarity, label overlap, sample redundancy, and backtest overfitting. We identify several implementation-level limitations, including a fractional differencing trade-off between stationarity and memory preservation, a simplified PnL calculation that may not reflect true strategy economics, and the fragility of single-asset, single-month validation. The walk-forward validation yields a net Sharpe ratio of 1.27, which we interpret as a stylized diagnostic rather than a validated trading result, due to a simplified PnL calculation that may not reflect true strategy economics.

All performance metrics in this report—including Sharpe ratios, PSR, DSR, and cumulative returns—are derived from a stylized correctness-based PnL encoding rather than actual realized returns, and should be interpreted as relative diagnostics, not validated economic results. See Section 8.3 for details.

Keywords: financial machine learning, fractional differentiation, meta-labeling, dollar bars, triple-barrier method, deflated Sharpe ratio, order flow imbalance, cryptocurrency microstructure

1. Introduction

The application of machine learning to financial markets presents challenges that are qualitatively distinct from those encountered in conventional ML settings. Financial data are non-stationary, serially correlated, and characterized by low signal-to-noise ratios. Standard cross-validation techniques leak information through temporal overlap. Labels derived from future returns are path-dependent and overlap across samples. Backtests are susceptible to overfitting through multiple testing. López de Prado's *Advances in Financial Machine Learning* (2018) addresses each of these issues with purpose-built techniques that collectively form a coherent pipeline for financial model development.

This report has two objectives. First, we provide a structured exposition of the book's core methodological contributions, organized around the pipeline stages that a quantitative researcher would follow in practice. Second, we present a critical implementation study that applies these techniques to high-frequency cryptocurrency trade data from Binance, using a Jupyter notebook that covers the full pipeline from raw tick data to walk-forward validation. Throughout, we emphasize where the implementation faithfully follows the book's prescriptions, where simplifications were made, and where the results should be interpreted with caution.

The choice of cryptocurrency data is motivated by accessibility: Binance provides freely downloadable aggregated trade data at the tick level, enabling the construction of true information-driven bars rather than relying on pre-aggregated OHLCV data. SOL/USDT was selected for its high trading volume and pronounced volatility during October 2023, a period that included significant price movements in the Solana ecosystem.

2. Conceptual Framework

2.1 The Failure of Standard Approaches

López de Prado argues that the standard machine learning workflow—random sampling, time-based bars, fixed-horizon labels, standard cross-validation, and single backtests—is systematically flawed when applied to financial data. Time-based bars sample at fixed intervals regardless of market activity, producing over-sampling during quiet periods and under-sampling during active ones. Fixed-horizon labels ignore the path-dependent nature of price movements. Standard K-fold cross-validation leaks information when labels span multiple observations. A single backtest cannot distinguish genuine skill from luck across many trials.

2.2 Information-Driven Bars

The book proposes sampling bars based on the cumulative flow of information rather than calendar time. Dollar bars, which sample whenever cumulative dollar volume exceeds a threshold, are a key example. By construction, dollar bars sample more frequently during high-activity periods and less during quiet ones, producing returns that are more likely to be independent and identically distributed. Additionally, the asynchronous sampling reduces the autocorrelation structure that plagues time-based bars.

2.3 Fractional Differentiation

A central tension in financial feature engineering is that stationarity (required by most ML algorithms) is typically achieved through integer differencing, which destroys the memory structure of the original series. Fractional differentiation generalizes differencing to non-integer orders $d \in (0, 1)$, allowing the series to achieve stationarity while retaining a controlled degree of memory. The weights for a fractional difference of order d are computed recursively:

$$w_k = -w_{k-1} \frac{d - k + 1}{k}$$

with $w_0 = 1$. The differenced series is then a weighted sum of past values. A threshold on the weight magnitude controls the effective lookback window. The key insight is that for many financial series, $d < 1$ suffices for stationarity while preserving substantially more memory than $d = 1$.

2.4 Triple-Barrier Labeling

Rather than labeling based on fixed-horizon returns, the triple-barrier method places upper and lower barriers (take-profit and stop-loss) and a vertical barrier (time limit) around the entry price. The label is determined by which barrier is hit first: +1 for the upper barrier, -1 for the lower, and 0 for the vertical (timeout). The barrier widths are typically set as multiples of the rolling volatility, making them adaptive to changing market conditions. This path-dependent labeling captures the real-world decision structure of trading and produces labels that are more informative than simple sign-of-return classifications.

2.5 Sample Uniqueness and Concurrency

When labels span multiple observations (as with triple-barrier labels), training samples are not independent. López de Prado introduces the concept of *concurrency*—the number of labels that are "active" at any given time point—and proposes weighting each sample by the inverse of its average concurrency. This *uniqueness* weighting ensures that periods with many overlapping labels do not dominate the training process, reducing the effective redundancy in the dataset.

2.6 Purged Cross-Validation

Standard K-fold cross-validation leaks information when labels overlap across the train-test boundary. The book's *PurgedKFold* addresses this by removing training samples whose label windows overlap with any test sample. An additional *embargo* period after the test set prevents leakage from short-term serial correlation. This is a necessary correction for any time-series classification task with path-dependent labels.

2.7 Meta-Labeling

Meta-labeling is a two-layer architecture that separates the direction decision from the bet decision. A primary model predicts the direction (long or short), and a secondary meta-model predicts whether the primary model's signal will be correct. This decomposition allows the researcher to train the primary model for recall (capturing all potential opportunities) and the meta-model for precision (filtering out false positives). The meta-model's output probability serves as a confidence measure that can be used for bet sizing.

2.8 Bet Sizing and Position Management

The book advocates for dynamic position sizing based on the meta-model's confidence. Rather than taking a fixed position whenever a signal is generated, the position size is scaled by the predicted probability of success. This transforms a binary trading decision into a continuous one, potentially improving risk-adjusted returns by allocating more capital to high-confidence signals and less to marginal ones.

2.9 The Deflated Sharpe Ratio

López de Prado introduces the Probabilistic Sharpe Ratio (PSR) and Deflated Sharpe Ratio (DSR) to address the multiple-testing problem in backtesting. The PSR adjusts the observed Sharpe ratio for skewness, kurtosis, and sample length, computing the probability that the true Sharpe ratio exceeds a benchmark. The DSR further adjusts for the number of trials conducted, recognizing that the maximum Sharpe ratio across multiple backtests is inflated. These metrics provide a more honest assessment of whether observed performance is likely to be genuine or an artifact of selection bias.

3. Data and Experimental Design

3.1 Data Source

The primary dataset consists of Binance SOL/USDT aggregated trade data (aggTrades) for October 2023, downloaded from Binance Data Vision. The raw data contains 3,105,897 individual trade records, each with price, quantity, timestamp, and a flag indicating whether the buyer was the maker (i.e., whether the trade was a taker buy or taker sell).

3.2 Dollar Bar Construction

Dollar bars are constructed by accumulating trades until the cumulative dollar value (price $\times$ quantity) exceeds a threshold of \$1,000,000. At each bar, the following fields are recorded: open, high, low, close prices; cumulative volume; cumulative dollar value; and the net aggressive buy dollar value (order flow imbalance, OFI). The OFI is computed by assigning a positive sign to taker-buy trades and a negative sign to taker-sell trades, then summing the dollar values within each bar.

This process yields 4,549 dollar bars from the 3.1 million ticks, a compression ratio of approximately 683:1.

3.3 Feature Engineering

The primary feature is the fractionally differenced cumulative order flow imbalance (OFI). The cumulative OFI is first standardized (z-score), then subjected to fractional differencing via the fixed-width window method (FFD). A grid search over $d \in \{0.1, 0.2, \dots, 1.0\}$ evaluates each candidate's ADF p-value and correlation with the original series.

Additional features are constructed for the meta-model:

- **Volatility:** 20-bar rolling standard deviation of close-price returns
- **High-low spread:** (high – low)/close per bar
- **Volume change rate:** Percentage change in bar volume
- **Return autocorrelation:** 20-bar rolling first-order autocorrelation
- **Volatility of volatility:** Rolling standard deviation of the rolling volatility
- **OFI volatility:** 20-bar rolling standard deviation of the stationary OFI

3.4 Labeling

Triple-barrier labels are generated with a maximum holding period of 20 bars, take-profit and stop-loss multipliers of $2.0\times$ the rolling volatility (EWM span=100). The resulting label distribution is:

Label	Count	Percentage
+1 (Take-profit)	2,370	51.99%
-1 (Stop-loss)	2,089	45.82%
0 (Timeout)	100	2.19%

The slight positive skew is consistent with the general upward drift in SOL/USDT during October 2023.

3.5 Sample Weighting

Concurrency-based uniqueness weights are computed for all labeled samples. The mean uniqueness weight is 0.0477, indicating substantial overlap among labels (an average concurrency of approximately 21 simultaneous active labels). This high overlap underscores the importance of the weighting scheme.

3.6 Model Architecture

The implementation uses a dual-layer architecture:

- **Primary model:** Random Forest Classifier (100 trees, max depth 4–5) predicting triple-barrier direction
- **Meta-model:** Random Forest Classifier (100 trees, max depth 3) predicting whether the primary model's prediction is correct, with isotonic probability calibration

4. Methodology and Implementation Walkthrough

4.1 Stage 1: Tick Data Acquisition and Dollar Bar Construction

The pipeline begins by fetching raw aggTrades from the Binance API (for a small live sample) and downloading a full month of historical data from Binance Data Vision. The data is stored in Parquet format for efficient retrieval. Dollar bars are generated by iterating through the tick array and accumulating dollar volume until the threshold is reached.

A key implementation detail is the computation of order flow imbalance (OFI) at the bar level. Each trade is classified as aggressive buy or aggressive sell based on the `is_buyer_maker` flag. The net aggressive buy dollar value within each bar serves as a microstructure signal capturing the directional pressure of informed order flow.

4.2 Stage 2: Normality Comparison

The notebook compares the distributional properties of returns from traditional 1-minute time bars versus dollar bars. The Jarque-Bera test results show that dollar bar returns exhibit excess kurtosis closer to zero (-0.087) compared to time bar returns (-0.646), and neither rejects normality at conventional levels. While both p-values are high (0.83 and 0.78 respectively), the kurtosis of dollar bars being nearer to the Gaussian benchmark of zero is consistent with the theoretical prediction that information-driven sampling produces returns closer to IID.

4.3 Stage 3: Fractional Differencing

The FFD grid search reveals an important practical tension. On the small live-tick sample (Cell 2), $d = 1.0$ achieves stationarity (ADF p-value 1.42×10^{-8}) while retaining 52.44% of the original memory—a promising result. However, when applied to the full monthly dataset (Cell 12), the grid search shows that only $d = 1.0$ passes the ADF test, and at that point the correlation with the original series drops to -0.016, indicating near-complete memory destruction. Lower values of d (0.1–0.9) retain near-perfect correlation but fail to achieve stationarity (all p-values > 0.48).

The implementation resolves this by manually selecting $d = 0.4$, which preserves memory but does not pass the ADF test. This is a design trade-off: the author acknowledges that tree-based models can tolerate mild non-stationarity, and the memory preservation is considered more valuable than strict stationarity for the downstream classification task. This decision is defensible in practice but deviates from the book's recommendation that stationarity should be verified before using a series as an ML feature.

4.4 Stage 4: OFI Predictability Test

An OLS regression of next-bar returns on the current OFI signal (with HAC-corrected standard errors) yields a coefficient of 0.0002 with a p-value of 0.315, indicating no statistically significant predictive relationship at the bar level. This result is based on a very small sample (9 observations from the live-tick demonstration) and should not be interpreted as a definitive assessment of OFI's predictive power. The result does highlight the difficulty of finding predictive signals at the microstructure level, even with information-driven sampling.

4.5 Stage 5: Triple-Barrier Labeling and Sample Weights

The triple-barrier labeling produces a roughly balanced dataset (52% long, 46% short, 2% timeout). The sample uniqueness weights, with a mean of 0.0477, reflect the high degree of label overlap inherent in a 20-bar holding period. These weights are carried forward into all subsequent model training, ensuring that overlapping samples do not artificially inflate the model's apparent performance during training.

4.6 Stage 6: Purged K-Fold Cross-Validation

A custom `PurgedKFold` class implements the embargo-based purging described in AFML Chapter 7. With 5 folds and a 1% embargo, the cross-validation yields a mean accuracy of 0.5251 (standard deviation 0.0510). This is modestly above the 50% random baseline, indicating weak but non-trivial predictive signal. The fold-to-fold variation (0.4464 to 0.5877) suggests regime sensitivity.

It should be noted that this implementation uses a simplified embargo approach rather than the full event-overlap purging using true label end times (t_1) described in the book. The `PurgedKFold` class excludes samples near the test set boundaries but does not check whether individual label windows (which may extend up to 20 bars beyond the entry point) overlap with the test set. This is a known simplification that could permit some leakage.

4.7 Stage 7: Meta-Labeling

The meta-labeling architecture evolves through several iterations:

1. **Initial attempt (Cell 15):** A simple sign-based primary model ($\text{OFI} > 0 \rightarrow \text{long}$) with a single-feature meta-model. The meta-model fails to learn anything useful (precision for class 1 is 0.00), and no trades pass the confidence threshold of 0.6.
2. **Multi-feature expansion (Cell 16):** Adding volatility, high-low spread, and volume change features. The meta-model still produces degenerate predictions (all-zero for class 1), with no trades passing any threshold.
3. **ML-based primary model (Cell 19–20):** Replacing the sign-based heuristic with a Random Forest primary model trained on the same features. The primary model achieves 58% baseline accuracy. Cross-validated predictions on the training set are used to generate meta-labels, and the meta-model is trained on these. At a threshold of 0.50, the filtered win rate remains at 58% (no filtering), and higher thresholds monotonically decrease the win rate.
4. **Probability calibration (Cell 21):** Applying isotonic regression calibration to the meta-model. This produces more meaningful probability distributions. At a threshold of 0.58, only 8.2% of trades are retained, but the win rate rises to 65.31%. At 0.60, retention drops to 4.0% with a 58.33% win rate. This demonstrates the precision-recall trade-off: higher confidence thresholds improve win rates but dramatically reduce trade frequency.

4.8 Stage 8: Position Sizing

The notebook implements and compares several position sizing schemes:

- **Linear sizing:** $\text{Position} = \text{direction} \times (\text{probability} - 0.5) \times 2$
- **Non-linear sizing with threshold:** $\text{Position} = 0$ if $\text{probability} \leq 0.52$, else normalized probability raised to the 0.7 power
- **Regime-aware sizing:** Position based on market regime features (return autocorrelation, volatility of volatility, OFI volatility)

- **Continuous scaling:** Linear mapping from 0.5–1.0 probability to 0.2–1.0 position weight, avoiding the distributional distortion from hard thresholds

The regime-aware meta-model (Cell 26) achieves a baseline win rate of 57.72% on the test set. At a threshold of 0.60, retention drops to 20.3% but the win rate rises to 66.94%.

4.9 Stage 9: Performance Evaluation

All performance metrics below are derived from a simplified PnL encoding (see Section 8.3) and should be interpreted as stylized diagnostics rather than validated strategy economics. Relative comparisons between strategies may still be informative.

Performance metrics are computed for the relaxed threshold dynamic position strategy (threshold 0.56):

Metric	Fixed Position	Dynamic Position (> 0.56)
Total Cumulative Return	92.00	21.16
Sharpe Ratio	2.48	2.49
Maximum Drawdown	-43.00	-9.85
Mean Per-Trade PnL	0.154	0.036
Per-Trade Volatility	0.989	0.226

The dynamic position strategy achieves a comparable Sharpe ratio with substantially lower drawdown and volatility, illustrating the risk-management benefits of confidence-based sizing.

4.10 Stage 10: Overfitting Defense

The PSR and DSR are computed for the dynamic position strategy:

Metric	Pre-Cost	Post-Cost	Continuous Scaling
Estimated Sharpe	2.76	2.71	2.42
Observations	490	505	548
Skewness	-0.23	-0.22	-0.27
Kurtosis	1.28	1.31	1.16
PSR (benchmark = 0)	100%	100%	100%
Expected Max SR	0.89 (15 trials)	0.95 (20 trials)	0.95 (20 trials)
DSR	100%	100%	100%

Both PSR and DSR are at 100%, suggesting that the observed Sharpe ratio is highly unlikely to be a chance artifact even after accounting for multiple testing. However, these results should be interpreted with caution due to the simplified PnL calculation (discussed in Section 8.3), the user-specified parameters (trials variance = 0.25, number of trials = 15–20), and the fact that the DSR's multiple-testing correction assumes the number of trials is known and fixed, which is difficult to guarantee in practice.

4.11 Stage 11: Walk-Forward Validation

A 5-fold walk-forward validation (expanding training window) produces:

- Total out-of-sample observations: 2,396
- Total cumulative net PnL (after 10 bps friction cost): 101.25
- Walk-forward net Sharpe ratio: 1.27

The walk-forward Sharpe ratio of 1.27 is substantially lower than the in-sample Sharpe of 2.71, reflecting the expected degradation from in-sample to out-of-sample performance.

Nevertheless, it remains above 1.0, which is often considered a minimum threshold for a viable strategy.

5. Results Summary

5.1 Dollar Bar Properties

The dollar bar construction successfully compresses 3.1 million ticks into 4,549 bars. The OFI within each bar provides a microstructure signal that captures the net directional pressure of aggressive order flow. The returns from dollar bars exhibit lower kurtosis than time bars, consistent with the theoretical expectation of more IID-like behavior.

5.2 Fractional Differencing Trade-off

The grid search reveals a fundamental tension: no value of $d < 1.0$ achieves ADF stationarity on the monthly OFI series, while $d = 1.0$ destroys nearly all memory. The pragmatic choice of $d = 0.4$ preserves memory at the cost of stationarity, a trade-off that is acknowledged but not fully resolved. This highlights the difficulty of applying fractional differencing to cumulative order flow series, which may have different memory properties than price series.

5.3 Meta-Labeling Effectiveness

The meta-labeling architecture demonstrates a clear precision-recall trade-off. The primary model's 58% baseline win rate can be improved to 65–67% at high confidence thresholds, but only by retaining 4–20% of trades. The isotonic probability calibration is essential for producing meaningful confidence scores—without it, the meta-model's raw probabilities are compressed into a narrow range that provides little discriminative power.

5.4 Position Sizing Benefits

Dynamic position sizing based on meta-model confidence produces a comparable Sharpe ratio to fixed sizing while dramatically reducing maximum drawdown (from -43 to -9.85) and per-trade volatility (from 0.99 to 0.23). This is the most compelling empirical result in the notebook and aligns with the AFML thesis that bet sizing is as important as directional prediction. However, these metrics are based on the simplified PnL encoding and should be treated as relative diagnostics (see Section 8.3).

5.5 Walk-Forward Validation

The walk-forward Sharpe ratio of 1.27, while lower than the in-sample metrics, provides the most honest estimate of out-of-sample performance. The gap between in-sample (2.71) and out-of-sample (1.27) Sharpe ratios is consistent with overfitting, though the DSR analysis suggests the performance is unlikely to be entirely spurious. This metric is also subject to the PnL encoding caveat discussed in Section 8.3.

6. Book-to-Code Mapping

The following table maps the book's chapters to the notebook's implementation, indicating which AFML concepts are covered and noting key deviations:

AFML Chapter	Book Topic	Notebook Cells	Implementation Status	Key Caveat
Ch. 2: Financial Data Structures	Bars (tick, volume, dollar, info-driven)	0, 6–9	Implemented (dollar bars from raw <code>aggTrades</code>)	OFI computed at bar level; no volume or tick bars for comparison
Ch. 3: Labeling	Triple-barrier method, meta-labeling	9, 15–16, 19–21	Implemented (triple-barrier + meta-labeling)	Simplified barrier width; meta-labeling uses CV predictions, not book's exact procedure
Ch. 4: Sample Weights	Concurrency, uniqueness, return attribution	10	Implemented (concurrency + uniqueness weights)	Does not implement return attribution or time decay
Ch. 5: Fractionally Differentiated Features	FFD, stationarity-memory trade-off	2, 11–13	Implemented (FFD with grid search)	No $d < 1.0$ achieves ADF stationarity on monthly data; $d=0.4$ used pragmatically
Ch. 6: Ensemble Methods	Random Forest, bagging	14–20	Implemented (RF throughout)	No comparison with boosting or bagging analysis
Ch. 7: Cross-Validation	Purged K-Fold, embargo	14	Partially implemented	Simplified embargo, not full t1-based purging; no CPCV
Ch. 8: Feature Importance	MDI, MDA, SFI	17, 25	Partially implemented	Only RF <code>feature_importances_</code> used; no MDI/MDA/SFI
Ch. 9: Hyper-Parameter Tuning	Grid/randomized search CV	18	Not implemented	Manual hyperparameter adjustment only
Ch. 10: Bet Sizing	Probability-based sizing	22–23, 27–28, 32	Implemented (multiple sizing schemes)	Simplified; no averaging of active bets or limit price logic
Ch. 11: Dangers of Backtesting	Backtest overfitting awareness	N/A	Implicitly addressed	No explicit strategy selection framework
Ch. 12: Backtesting via CV	Walk-forward, CPCV	33	Partially implemented	Walk-forward only; no CPCV
Ch. 13: Backtesting on Synthetic Data	Monte Carlo, trading rules	Not implemented	Not covered	—

AFML Chapter	Book Topic	Notebook Cells	Implementation Status	Key Caveat
Ch. 14: Backtest Statistics	PSR, DSR, efficiency	30–32	Implemented (PSR + DSR)	Parameters user-specified; PnL encoding affects results
Ch. 15: Understanding Strategy Risk	Probability of strategy failure	29	Partially implemented	Sharpe and drawdown computed; no strategy failure probability
Ch. 16: ML Asset Allocation	Hierarchical Risk Parity	Not implemented	Not covered	—
Ch. 17: Structural Breaks	CUSUM, explosiveness tests	Not implemented	Not covered	—
Ch. 18: Entropy Features	Shannon entropy, Lempel-Ziv	Not implemented	Not covered	—
Ch. 19: Microstructural Features	Price sequences, strategic/sequential trade models	0, 7, 9	Partially implemented	OFI computed; no VPIN, Kyle's lambda, or Amihud illiquidity
Ch. 20: Multiprocessing	Vectorization, parallelism	7, 9	Implicitly addressed	Numpy vectorization used; no explicit multiprocessing framework
Ch. 21: Brute Force & Quantum	Combinatorial optimization	Not implemented	Not covered	—
Ch. 22: HPC Forecasting	Regulatory, HPC infrastructure	Not implemented	Not covered	—

The notebook covers approximately 10 of the 22 chapters substantively, with partial coverage of several more. The primary gaps are in structural breaks (Ch. 17), entropy features (Ch. 18), advanced microstructural features beyond OFI (Ch. 19), synthetic data backtesting (Ch. 13), and hierarchical risk parity (Ch. 16). These omissions are understandable for a beginner-level implementation but represent opportunities for extension.

7. Feature Importance Analysis

The feature importance comparison between the primary and meta-models reveals an interesting pattern:

Feature	Primary Model	Meta-Model
OFI Stationary	0.582	0.446
Volatility	0.263	0.433
HL Spread	0.053	0.063
Volume Change	0.102	0.058

The primary model relies heavily on the OFI signal (58% importance), while the meta-model distributes its attention more evenly between OFI (45%) and volatility (43%). This suggests that

the meta-model is learning that the OFI signal's reliability varies with market conditions—precisely the type of conditional relationship that meta-labeling is designed to capture.

The standalone regime feature importance (Cell 17) shows OFI contributing 58%, volatility 22%, volume change 12%, and high-low spread 8%, confirming the dominance of the order flow signal in the primary model's decisions.

8. Critical Evaluation

8.1 Strengths of the Implementation

The notebook successfully demonstrates the full AFML pipeline from raw data to walk-forward validation, covering all major methodological contributions of the book. The use of genuine tick-level data and true dollar bar construction (rather than pre-aggregated OHLCV) is a notable strength. The iterative refinement of the meta-labeling architecture—from a degenerate single-feature model to a calibrated, regime-aware system—effectively illustrates the practical challenges of implementing these techniques. The inclusion of transaction costs in the final walk-forward validation adds a layer of realism often absent in academic implementations.

8.2 Fractional Differencing Limitations

The most significant methodological concern is the fractional differencing result. The book's framework assumes that a value of $d \in (0, 1)$ can be found that simultaneously achieves stationarity and preserves meaningful memory. For the cumulative OFI series on this dataset, no such d exists: stationarity requires $d = 1.0$ (which destroys memory), while any $d < 1.0$ fails the ADF test. The pragmatic choice of $d = 0.4$ is a reasonable engineering decision but represents a deviation from the book's theoretical framework. This issue may stem from the use of cumulative OFI (which has a unit root by construction) rather than the raw OFI or price series. Applying FFD to the non-cumulative per-bar OFI might yield more favorable stationarity-memory trade-offs.

8.3 PnL Calculation Concerns

The PnL calculation in the notebook uses a simplified scheme where `trade_outcome` is set to +1 if the primary model's prediction matches the true label and -1 otherwise. This is then multiplied by the position (which includes the directional sign). However, this encoding may produce directionally inconsistent results: a correct short prediction (`primary_side = -1`, `true_label = -1`) yields `trade_outcome = +1`, and the PnL becomes $(-1) \times (+1) = -1$, which is a loss for a correct prediction. This appears to be an error in the PnL logic, or at minimum a non-standard encoding that conflates prediction correctness with directional returns.

As a result, all performance metrics derived from this PnL—including the Sharpe ratios, maximum drawdowns, PSR, and DSR—should be interpreted as stylized diagnostics rather than validated strategy economics. The relative comparisons between fixed and dynamic position sizing may still be informative (as both use the same encoding), but the absolute levels of the metrics should not be taken at face value.

8.4 Purged K-Fold Simplifications

The PurgedKFold implementation uses a fixed embargo window rather than true event-overlap purging. The book specifies that training samples should be purged if their label end times (t_1) fall within the test set's time range. The notebook's implementation approximates this with a fixed number of samples around the test boundary, which may not fully eliminate leakage for labels with long holding periods. Additionally, the implementation does not use Combinatorial Purged Cross-Validation (CPCV), which the book recommends as a more robust alternative.

8.5 Threshold Selection Bias

Several probability thresholds (0.52, 0.56, 0.60) are tested and compared within the notebook, and the "best" performing threshold is selected for subsequent analysis. This selection-after-observation introduces a form of data snooping that the DSR is designed to address, but the DSR's parameters (number of trials, trials variance) are user-specified and may not accurately reflect the true number of effective trials conducted during the notebook's development.

8.6 Annualization Assumptions

The Sharpe ratio is annualized using $\sqrt{252}$, which assumes approximately 252 trading days per year. However, dollar bars are not time-based, and the number of bars per day varies with market activity. The appropriate annualization factor for event-time bars is not $\sqrt{252}$ but rather $\sqrt{\text{average bars per year}}$, which could be substantially different. This inflates or deflates the reported Sharpe ratios depending on the average bar frequency.

8.7 Single Asset and Single Month

The entire study uses a single cryptocurrency pair (SOL/USDT) over a single month (October 2023). This severely limits external validity. The label distribution shows a slight positive skew (52% long vs. 46% short), which may reflect the market conditions during this specific period. Results may not generalize to other assets, other time periods, or other market regimes. The walk-forward validation, while providing some out-of-sample evidence, still operates within the same month and asset.

8.8 Transaction Cost Model

The transaction cost model assumes a fixed 10 bps per unit of position change, which is a rough approximation. Real trading involves exchange fees (typically 0.1% for taker orders on Binance), slippage that depends on order size and market depth, and potentially funding costs for leveraged positions. The simplified cost model may understate the true cost of frequent position changes, particularly for the continuous scaling strategy that adjusts position size on every bar.

8.9 Cryptocurrency Microstructure Specificity

The OFI signal derived from Binance aggTrades may have different properties than order flow imbalance in equity markets. Cryptocurrency markets operate 24/7, have different market microstructure (no designated market makers, different order types), and are subject to different regulatory and behavioral factors. The results should not be assumed to transfer to equities, futures, or other asset classes without further validation.

9. Future Work

9.1 Methodological Improvements

1. **True event-end-time purging:** Implement `PurgedKfold` with actual t_1 label end times to eliminate all label overlap leakage. Consider implementing CPCV for more robust backtesting.
2. **Fix PnL calculation:** Replace the simplified correctness-based PnL with actual realized returns. The position should be applied to the actual price change between entry and exit, not to a binary correctness signal.
3. **FFD on raw series:** Apply fractional differencing to the raw (non-cumulative) OFI or directly to the price series, which may yield better stationarity-memory trade-offs.
4. **Proper annualization:** Compute the average number of dollar bars per year and use $\sqrt{\text{bars per year}}$ for Sharpe ratio annualization.

9.2 Empirical Extensions

1. **Multi-asset validation:** Extend the pipeline to multiple cryptocurrency pairs (e.g., BTC/USDT, ETH/USDT) and traditional assets (e.g., S&P 500 futures) to assess cross-asset generalization.
2. **Multi-period validation:** Test across multiple months and years, including different market regimes (bull, bear, sideways), to assess temporal robustness.
3. **Realistic cost model:** Implement a transaction cost model that accounts for exchange fees, slippage as a function of order size and market depth, and funding costs.
4. **Feature stability analysis:** Compute feature importance across time windows to assess whether the relative importance of OFI, volatility, and other features is stable or regime-dependent.
5. **Alternative models:** Compare Random Forest with gradient boosting (XGBoost, LightGBM), neural networks, and other model classes to assess whether the meta-labeling benefits are model-specific.

10. Conclusion

This implementation study demonstrates the practical application of López de Prado's *Advances in Financial Machine Learning* framework to cryptocurrency microstructure data. The pipeline—from dollar bar construction through fractional differencing, triple-barrier labeling, purged cross-validation, meta-labeling, probability calibration, dynamic position sizing, and deflated Sharpe ratio analysis—illustrates the book's central thesis: financial machine learning requires purpose-built techniques at every stage, and the standard ML workflow is systematically inappropriate for financial data.

The empirical results, while suggestive, should be interpreted with appropriate caution. The fractional differencing trade-off, the simplified PnL calculation, the threshold selection bias, and the single-asset, single-month scope all limit the strength of the conclusions that can be drawn. The walk-forward Sharpe ratio of 1.27, adjusted for transaction costs, provides a reasonable but not definitive signal of out-of-sample performance, though this metric is subject to the PnL encoding caveat discussed in Section 8.3.

The most robust finding is qualitative rather than quantitative: the AFML framework's systematic approach to addressing financial ML's unique challenges—non-stationarity, label overlap, sample redundancy, and backtest overfitting—represents a genuine advance over naive approaches, and the meta-labeling architecture in particular offers a principled way to separate directional prediction from bet sizing decisions. The practical challenges encountered in this implementation, especially around fractional differencing and PnL calculation, highlight the gap between the book's theoretical framework and the realities of applied quantitative research—a gap that can only be bridged through careful, iterative, and critically self-aware implementation.

References

López de Prado, M. M. (2018). *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons. ISBN 978-1-119-48208-6.

Binance Data Vision. Historical market data downloads. Available at: <https://data.binance.vision/>

The implementation notebook accompanying this report: *Beginner's Attempt on Financial ML* (translated to English), containing the full pipeline code with 35 code cells covering data acquisition through walk-forward validation.